Hamal's Transportation App

High Level Design Doc

Objective

Develop a **mobile app** for Hamal's logistics department, in order to manage the organization's delivery missions efficiently.

Background and Overview

Currently deliveries are updated using a WhatsApp group-chat, only allowing sending and receiving information about who took the delivery and when it was completed.

We aim to improve the current workflow by developing a mobile app for both drivers and the logistic team's managers, emphasizing mission management, navigation and routes suggestions, real-time location sharing, providing personalized push-notifications for drivers based on their load capacity and location.

Requirements (User Stories)

Drivers can:

- Accept delivery requests
    - As a **driver**, I want to accept requests for delivery, so that I can support the organization.
- Mark deliveries as completed
    - As a **driver** I want to update the status of deliveries as completed, to notify the management about completed mission and let the organization update its mission list.
- Navigate to destination
    - As a **driver**, I want to navigate to my destination via the app, so that I can arrive to my destination
- Accept more deliveries on the go
    - As a **driver**, I want to receive delivery requests near me, to best utilize my gas expenses.
- Receive messages from the logistic team
    - As a **driver**, I want to receive messages about deliveries from the logistic team, so I can stay updated about changes to my deliveries.

Administrator can:

- View tasks
    - As an **administrator**, I want to track drivers' live location, so that:
        - I can track their progress.
        - I can confirm delivery goals are met.
        - I can let them know if there are nearby last-minute deliveries they may not see in time.
- View driver information
    - As an **administrator**, I want to see driver information, so that I can contact them.

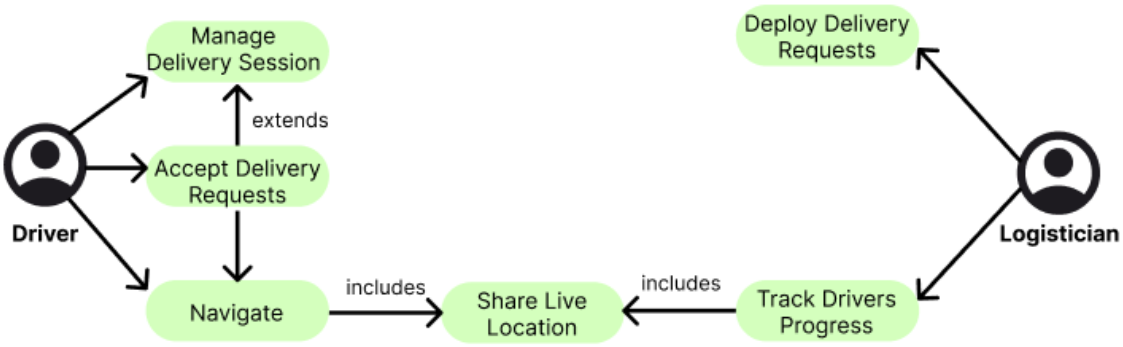
- Communicate with the drivers
    - As an **administrator** I want to send messages to the drivers, to provide them real time update or ask them questions on the go.

Logistics staff can:

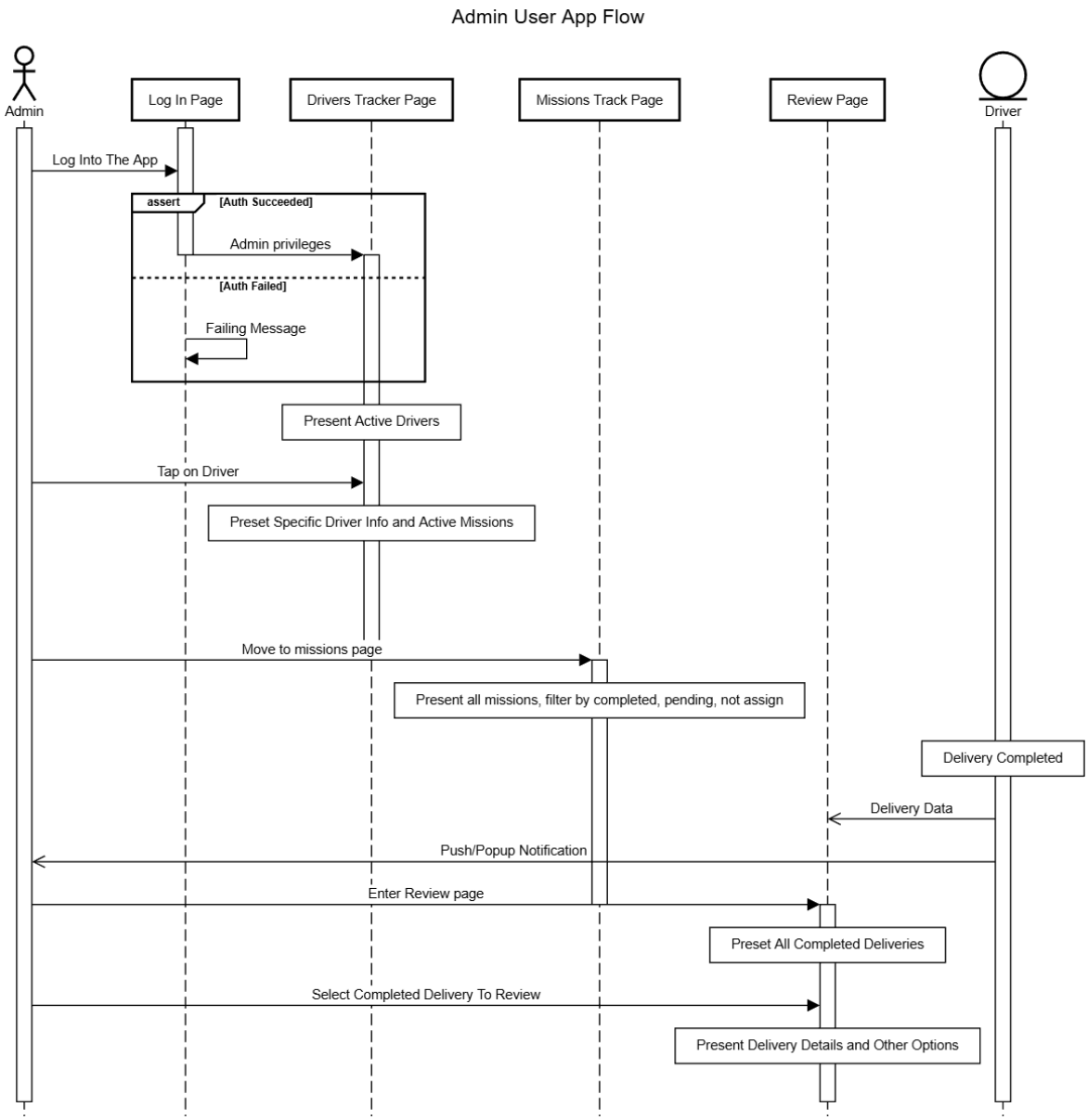
- Request deliveries
    - As a **logistician**, I want to deploy delivery requests to drivers, so that the drivers can accept them.
- Check on driver's progress
    - As a **logistician**, I want to track drivers' live location, so that:
        - I can confirm delivery goals are met
        - I can let them know if there are nearby last-minute deliveries they may not see in time.
- Specify delivery requirements (e.g.: haulers, large load capacity)
    - As a **logistician**, I want to add requirements to delivery, so that my drivers can only accept deliveries they can do.
- Communicate with the drivers
    - As a **logistician** I want to send messages to the drivers, to provide them real time update or ask them questions on the go.

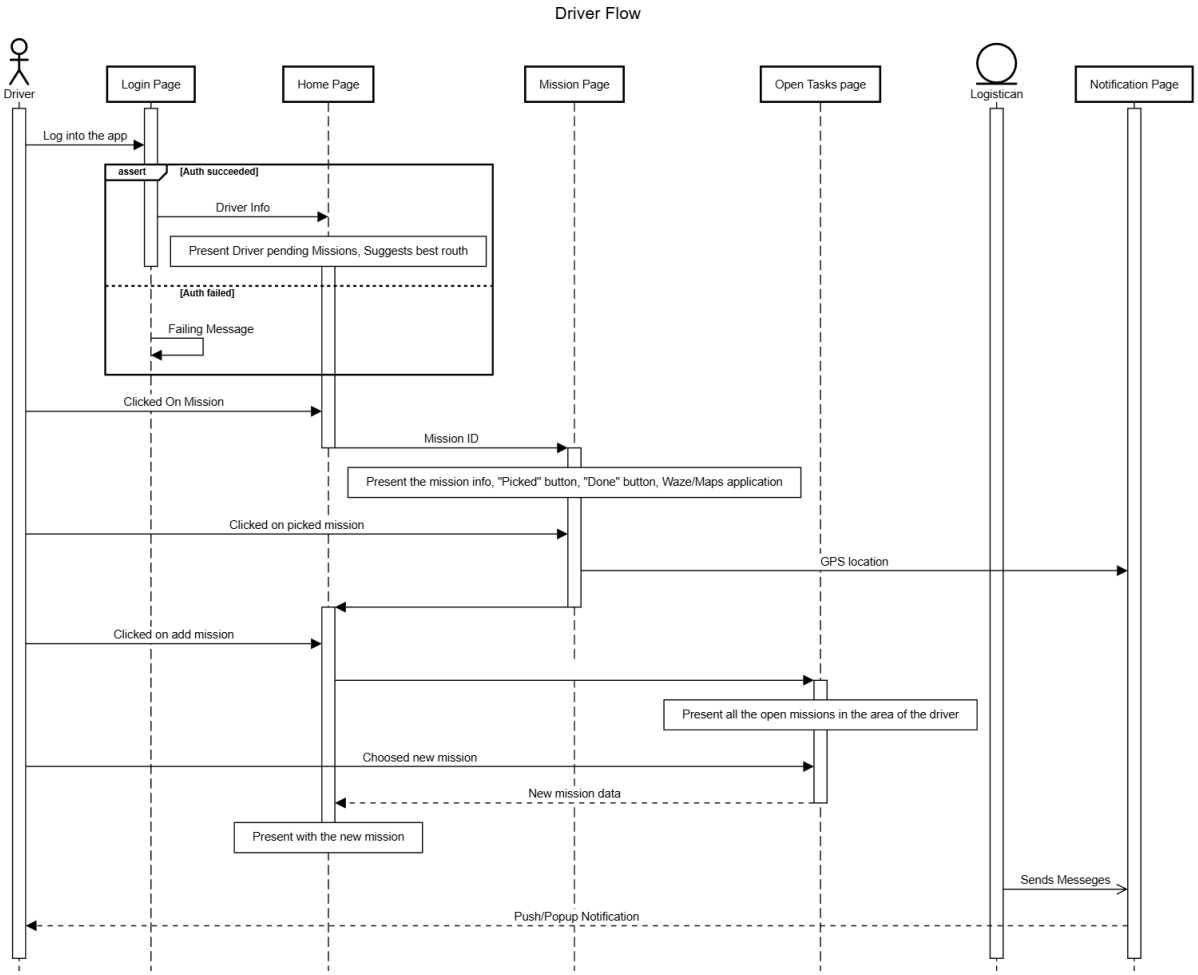
User Diagrams

Consider the following diagram for a simplified and concise specification of the application’s users' relationships:



App Flow


Admin User App Flow
Admin
Log In Page
Drivers Tracker Page
Missions Track Page
Review Page
Driver
Log Into The App
assert
[Auth Succeeded]
Admin privileges
[Auth Failed]
Failing Message
Present Active Drivers
Tap on Driver
Preset Specific Driver Info and Active Missions
Move to missions page
Present all missions, filter by completed, pending, not assign
Delivery Completed
Delivery Data
Push/Popup Notification
Enter Review page
Preset All Completed Deliveries
Select Completed Delivery To Review
Present Delivery Details and Other Options



Pages information

- **Login Page** – The first page users will see before logging in. Only authorized users can log in, after verification with the Hamal database. There will be separate login flows for drivers and for logistics.
- **Home Page** – This page serves as the main gateway to all other parts of the app. We chose to keep everything on one screen to make it easier for drivers to use. The screen will include buttons for *My Mission screen*, *Pick New Mission* screen and button for calling the Hamal desk. The screen also displays a list of the driver's active and upcoming missions. Each mission card includes details on the mission such as location, package description and contact information. Tapping a card opens the *Mission Screen*.
- **Mission Page** – This screen presents the full details of a specific mission.

It includes a button to update delivery status (chosen, picked up, delivered, cancelled, etc) and a button to lunch Waze for navigation to the mission's location.

- **Open Tasks Page** – This screen displays a list of available missions that the driver can choose from. Each mission card shows a brief summary, and tapping a card opens the Mission screen.

Tasks

Frontend:

- Create wireframes
- Layout UI mocks:
    - Log in screen
    - My tasks screen
    - New tasks screen
    - Task information screen
    - Menu screen
    - Map screen (*administrator*)
    - Driver phonebook screen (*administrator*)
    - Driver information card (*administrator*)
    - Prompt Waze drive
    - Contact information card
    - Task card
    - Contact HQ button (call)
- Push notifications:
    - New task available
    - Task updated
- Waze integration:
    - Export destination to Waze (start drive)
    - Get drive times/ distance for current tasks

Backend:

- Monday/ xrm integration (manage system)
- Driver session tracking
    - Active deliveries
    - Location tracking API
- Push notification service (to send notifications)
- Delivery assignment logic (to match deliveries to drivers)
- User authentication (to manage login)
- Simulate a database using python
    - Tasks:
        - Task id/ key
        - Contents (string/ list)
        - Source location (name + coordinates?)
        - Destination location
        - Sender contact information
        - Recipient contact information
        - Task status
        - Task load capacity requirements
        - Driver contacts information
        - Current location
        - Task taken/ not taken
    - Drivers/ Users:
        - Driver id/ key
        - Full name
        - Phone number
        - Car type (SUV/ truck/ sedan/ trailer)
        - Can / cannot lift heavy loads
    - Taken tasks:
        - Task id
        - Driver id

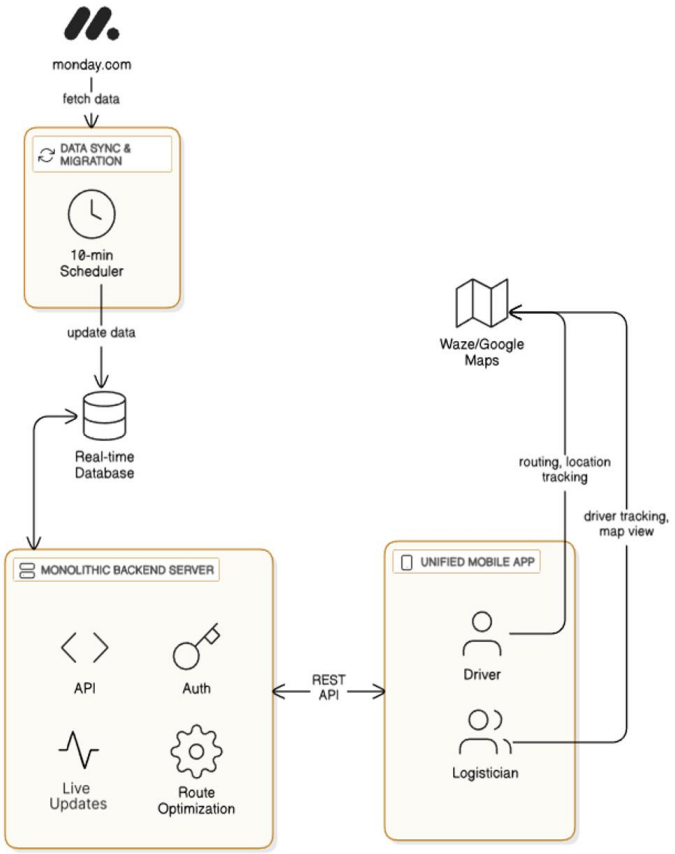
- After the MVP - Build a database for our application

Nice To Have

- Drives would be able to see their delivery history.
- Assign priority or deadlines to deliveries.
- View delivery reports and statistics.
- Ability to send messages and advertising to all the drivers.

Architecture & Tech Stack

High Level Architecture



We aim to follow a monolithic server-client approach: a single backend service acts as the system's source of truth, coordinating data from the internal database, mobile clients, and the monday.com integration layer. The mobile app communicates with the backend via well-defined REST APIs, while background workers manage long-running tasks such as synchronization, notification dispatching, and data ingestion from monday.com.

Real-time features, such as live location tracking and instant mission updates, are achieved using a combination of push notifications and lightweight WebSocket channels. For a system with ~100 users and limited concurrency, a monolithic architecture provides simplicity, strong consistency, and low operational overhead while avoiding over engineering.

Backend – Monolith Server

Python frameworks such as **Django/FastAPI** will be utilized to achieve a robust component that will handle the communication with our database and provide relevant data to the client side.

API Layer – REST API

RESTful API provides a clear, standardized interface that's easy to understand, test, and consume. Combined with WebSockets for real-time features, this hybrid approach offers the best balance of simplicity and functionality. REST handles CRUD operations while WebSockets handle live updates, avoiding REST's polling overhead for real-time data.

Frontend – MVVM Architecture, Role-Based UI

Flutter is chosen for its excellent cross-platform capabilities, native performance, beautiful UI, and hot reload for rapid development.

The MVVM (Model-View-ViewModel) architecture separates business logic from UI, making the code testable, maintainable, and scalable.

A single codebase with role-based UI reduces development time by 40-50% compared to building 2 separate apps, while maintaining a native look and feel.

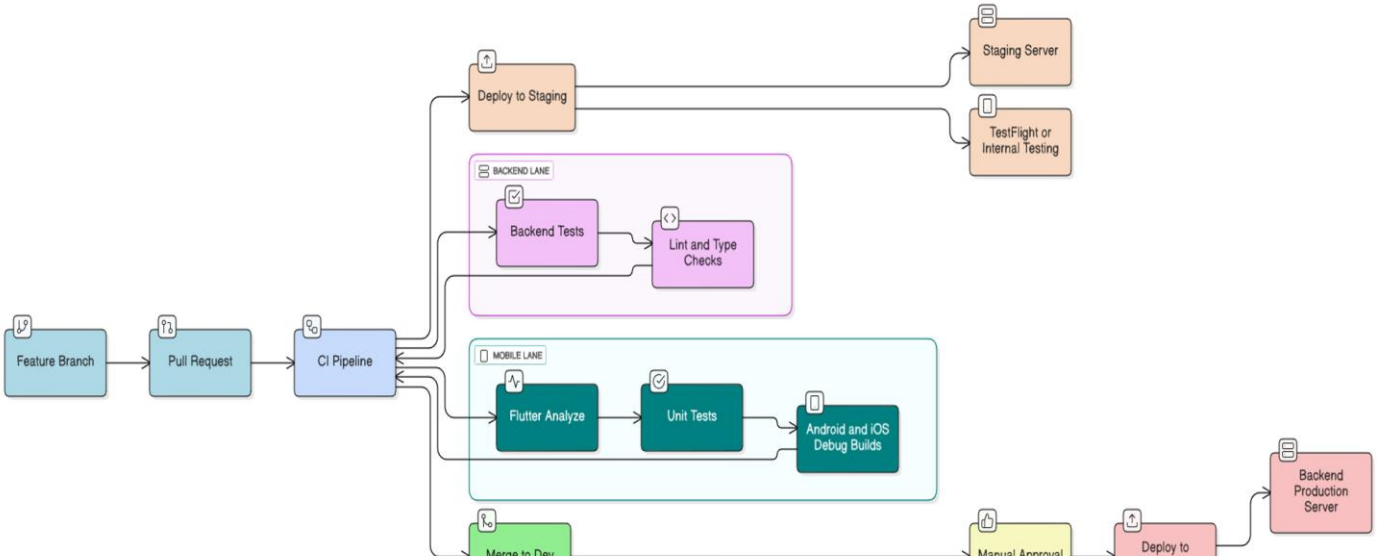
Data Layer

The data layer is centered on a **fully managed, real-time database service** that is highly scalable and ensures immediate data synchronization across all clients. This architecture is crucial for key functionalities, such as providing logisticians with instantaneous driver location updates and enabling drivers to see mission status changes without delay. A core benefit of utilizing this managed provider is the seamless integration with its proprietary push notification service, which is leveraged by the Backend Server to send personalized mission alerts directly to driver devices. Data integrity is maintained by housing distinct collections for users, driver locations, and missions.

Furthermore a dedicated **Data Sync Service** will handle the one-way ingestion process, polling or listening via webhooks to the external monday.com API to ensure new missions are pulled, validated, and inserted into the mission collection, establishing the external platform as the authoritative source for mission creation details.

CI/CD

The general flow of the CI/CD process:



Since our architecture aims to provide a server-client system, we need to separate concerns about integration and deployment of the different layers of the system.

We aim to utilize GitHub actions for this process, when a new feature is implemented, a pull request will be opened, upon approval the pipeline will start:

At the beginning, unit tests and other check will be executed on the backend unit (only if a change occurred in that unit).

Simultaneously, unit tests and other checks will be executed on the frontend unit, in a more advanced stage of this project we will try to add UI tests, then if all tests succeed, both Android and iOS build will be generated.

In the Dev branch, the stages will wait for manual approval and will be deployed afterwards.

We aim to also host a staging server that will play as a testing environment, all changes will be deploy to this server also, with the ability to rollback any change at any time.

Alternative Approaches

Monolith vs Microservices Backend Architecture

A monolith backend consolidates the all the logic - authentication, missions management, GPS tracking, notifications, and the integration with monday.com - into a single deployable unit. This approach is much simpler to develop, test and deploy and offers lower operational cost. In contrast, a microservices architecture separates the domains into independent service that communicate with each other. Althought it enables technological flexibility, it also introduces complexity in areas such as sercive discovery, inter-service communication and monitoring.

Given the relatively small scale of the system and the need for simplicity, the monolithic approach is more suitable to our case.

Separate Client Applications For Drivers And Logisticians

This approach develops two independent mobile apps - one for drivers and on for logisticians, instead of single unified application with Role-Based UI.
In this way each ap can be optimized for its specific workflow and allow reduce of permission and role logic within the UI.

While separate applications allow dedicated workflows and potentially simpler UX per role, they required more development and maintenance as they have two distinct codebases and deployment processes. In contrast, a single cross-platform application that presents different screens for driver and logistician offers a far more efficient solution.

Role-Based UI reduces development time, allows reuse of shared components such as authentication and navigation and simplifies long-term maintenance, while still ensuring that each user group sees only the functionality relevant to them. For these reasons, a unified application with role-based UI was chosen.

Cross Platform vs. Native Frontend Development

Native Frontend Development (such Swift or Kotlin) provides maximum performance and seamless access to device capabilities, but it is also requires two separate codebases which demand significantly higher development and maintenance efforts. For a small project like that, this is impractical.

Cross-platform solutions such as Flutter allows building a single application that runs on both iOS and Android, reducing development time while still matches the project requirements.

Although cross-platform frameworks introduce minor overhead and result in slightly larger app sizes, the advantages in development speed, uniform UI design, and maintainability make this approach a better fit.

API Layer Architecture

Restful API vs gRPC vs Graphql

The communication layer between the mobile application and the backend requires using API. Restful API provides simple and easy-to-document interface that aligns well with the needs of our system. It integrates naturally with Flutter and works reliably over mobile networks and matches the structure of external services like monday.com.

Although gRPC is fast and efficient, it's more suitable for internal microservices and lacks straightforward support in small mobile apps. Graphql also offers powerful data querying flexibility but introduces complexity in backend that are unnecessary for this project that has straightforward data flows.

To support real-time features such as live location updates and instant mission changes, as we think we will need to, Websockets complement the REST API and allow event-based updates without constant polling. This was the main reason that made us choose Rest over the other options.

Tech Stack Considerations

Python vs nodejs for backend

The decision of the backend implementing was guided by the questions of the the efforts in maintaining and the ease of integrating external services. Python uses frameworks such as Django or Fast API that offers a lot of things: clean development experience, provides support for data validation and API development, and a mature set of libraries. While Node.js offers excellent performance and package ecosystem, it tends to be more complex to maintain over time. For a project like ours, that centers on the background processing and integration with a managed database, Python offers a more stable solution.

Flutter vs React native

Although React native benefits from the large Javascript's ecosystem, it suffers from performance limitations due to its reliance on the Javascript bridge.

On the other hand, Flutter compiles to native machine code and ensure consistent UI behavior across devices which makes excellent developer productivity.

The fact that we need for clean and accessible UI for elderly drivers, as well as our willing to minimize long-term maintenance complexity, brings us to choose Flutter.

Managed database

For our app, we need a database that is always up to date and works well with many users at the same time. We also don't want to spend time setting up servers, fixing bugs, or taking care of backups. A managed database is a service that someone else runs for us, and we just use it through the internet.

Services like **Firebase Firestore** or **Supabase** are good examples. They automatically handle things like storing data, keeping it safe, backing it up, and making sure it works fast even if many people use the app at once. They also make it easy to send changes to all devices instantly - for example, when a logistician updates a mission or when a driver shares their location.

Using a managed database is the simplest and most practical choice, mainly because the need of real-time updates and the desire for easy maintenance.

Risks and Mitigation

Continues Monday.com Integration and Data Migration

The Hamal organization currently use Monday.com as their management system of the logistics department. They are planning to switch to a different management system but they still don't know which management system it will be.
we should minimize our dependence on the management system being Monday.com and implement the components of the project dealing with the connection to the management system as universal as possible. Moreover, we might need to implement creating deliveries from the logistician app profile until we know what management system the Hamal organization will use in the future.

GPS Tracking

The GPS tracking element of our project might present some difficulties.

The first one is battery drain. We are going to need real time location both during active delivery and when the drivers are not in active delivery. When the driver is in active delivery we need an accurate real time location so we will probe his GPS location in small intervals which will consume battery, we will have to find the correct intervals to balance between our need for accurate location and battery consumption.
We also need the driver's location when he is not currently delivering so we could send him customised push notifications about deliveries in his area. For this function we will probe its location in much bigger intervals.

Another issue could rise when the driver won't give his permission for using his GPS location. We don't have an alternative for active delivery's location but we can add a location to the driver's profile to customise his push notifications and available deliveries list.

Moreover, the GPS location can cause a security issue. The destinations of most of the deliveries are army bases and assembly areas that can be classified so me will need to disable GPS tracking when the driver is approaching the destination.

Target audience

The primary target audience of the app are the drivers, which are usually elderly people. We need make the app as simple and intuitive as we can so the elderly drivers could easily use the app. We also need to support accessibility options like bigger font size and bigger buttons.

Deployment Costs

The Hamal organization is a non-profit so we will need to keep the cost of the project as low as possible.

Database – we will use an existing database service that costs money so will need to make our database as efficient and small as we can to make it cheaper.

Server – we will probably run the backend server on the cloud so we will have to keep it simple and efficient so it will be cheaper to run it on the cloud.

Maintenance – the Hamal hires workers to maintain their systems so we need our project to be as simple and as an easy to maintain as we can, so this worker won't spend too much time maintaining it. This emphasizes the importance of making the code readable and easy to understand.

Integration with the second group

Parallel to us, there is another group working on an application for the same organization – the Hamal. Although the applications have different purposes and offer different services to the client (the Hamal), they have a few points of intersection, which might create conflicts between the two applications. These conflicts might occur because the two applications use the same components inside the Hamal. Additionally, after the deployment of our application and the end of our academic year, the technical maintenance team of the Hamal will be in charge of maintaining both applications,

meaning that we need to write both applications using the same programming languages and tools.

Proof of Concept

We are planning to use already existing services and platforms so there is no need for Proof of Concept.